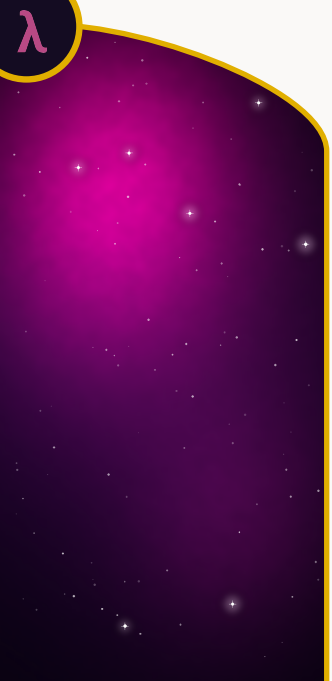


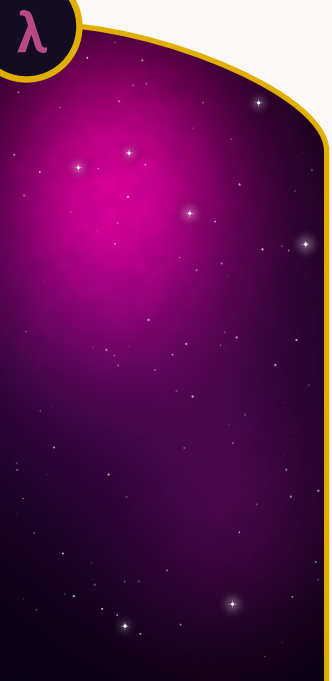
L1B: Runology

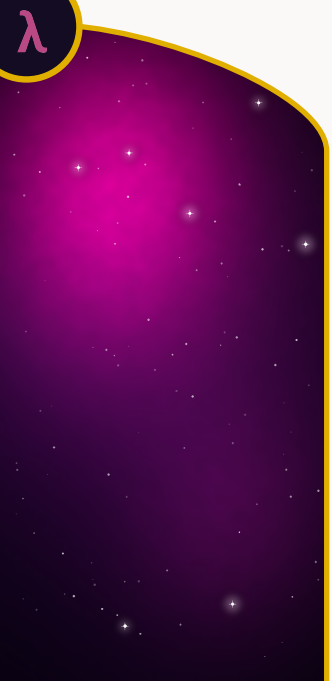
CS1101S: Programming Methodology

Boyd Anderson

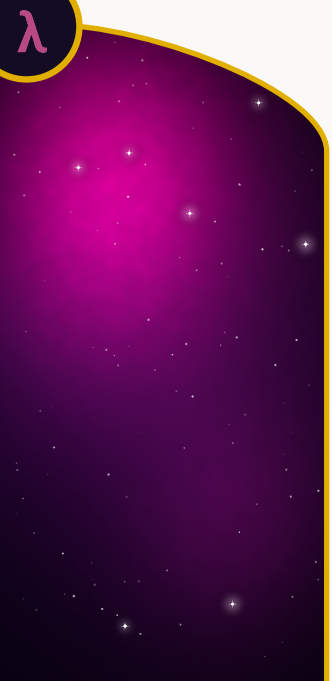
14th August, 2026

Version 1.0 — Built: 2026-08-18 23:40:27



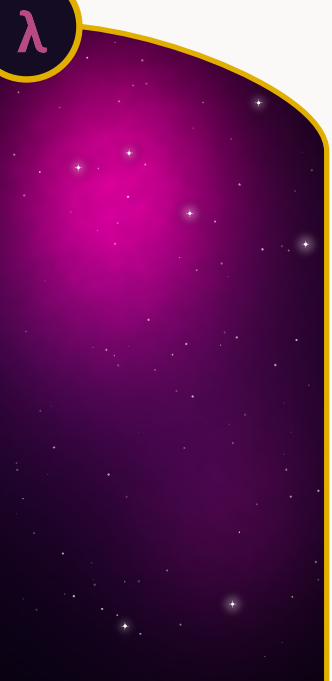


Announcements



Announcements

- This Monday/Tuesday 2-h **S**tudio session S2 on Elements of Programming

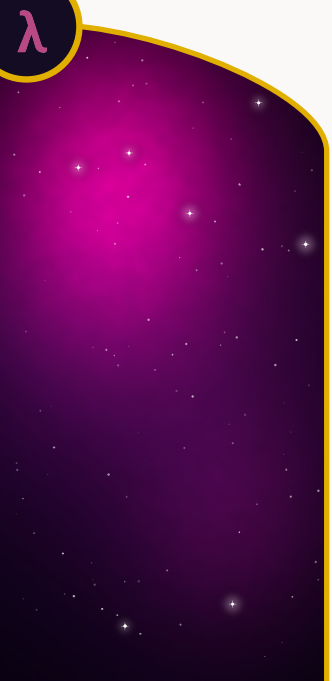


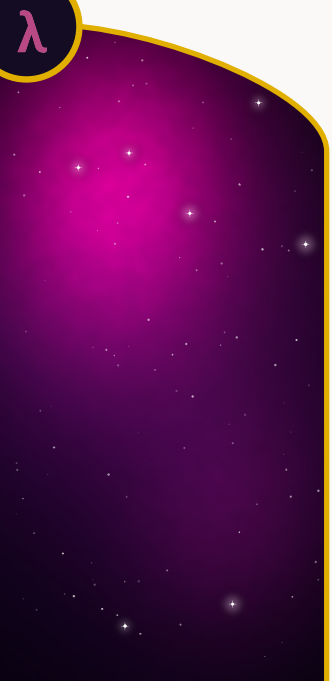
Announcements

- This Monday/Tuesday 2-h **S**tudio session S2 on Elements of Programming
- Don't worry if you have not been assigned a session. We will have an online option on Tuesday at 6pm-8pm.

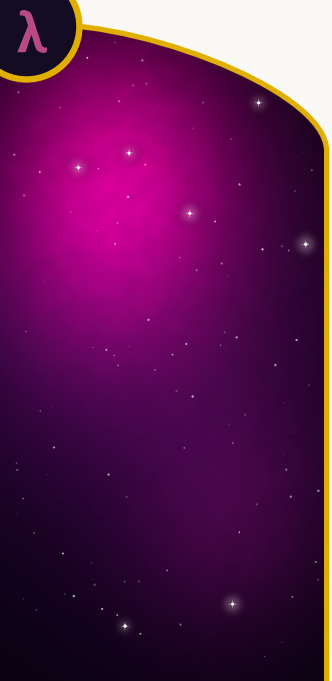
Announcements

- This Monday/Tuesday 2-h **S**tudio session S2 on Elements of Programming
- Don't worry if you have not been assigned a session. We will have an online option on Tuesday at 6pm-8pm.
- Don't forget about our Ed forums: https://edstem.org/us/courses/101145/discussion





First Mission: Rune Trials

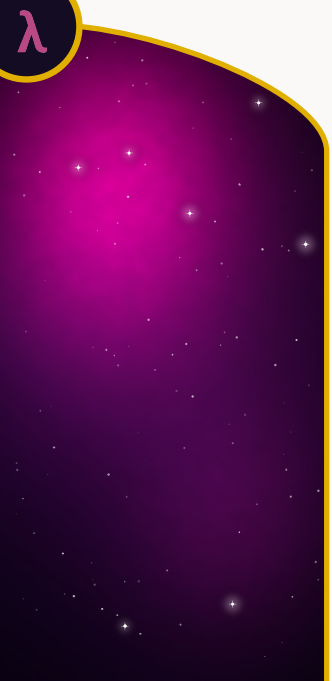


First Mission: Rune Trials

- ...will be released at 11:45 **today**.

First Mission: Rune Trials

- ...will be released at 11:45 **today**.
- Missions are submitted individually and carry significant XP.



First Mission: Rune Trials

- ...will be released at 11:45 **today**.
- Missions are submitted individually and carry significant XP.
- Path P1B will be released at 11:45 today.

First Mission: Rune Trials

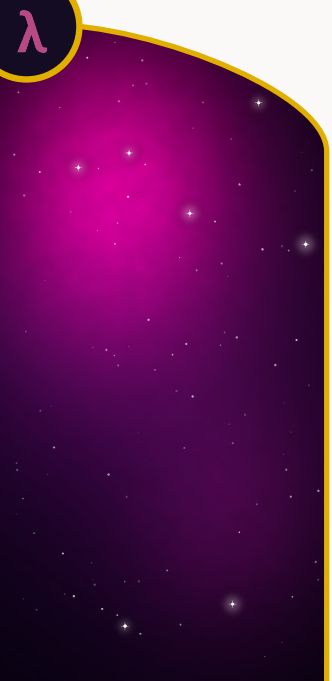
- ...will be released at 11:45 **today**.
- Missions are submitted individually and carry significant XP.
- Path P1B will be released at 11:45 today.
- You *can* discuss paths, missions, and quests with your friends and avengers.

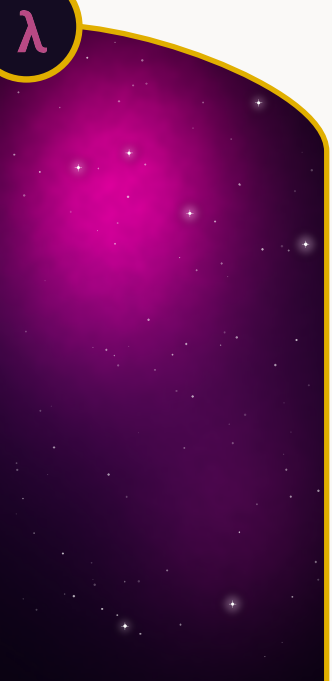
First Mission: Rune Trials

- ...will be released at 11:45 **today**.
- Missions are submitted individually and carry significant XP.
- Path P1B will be released at 11:45 today.
- You *can* discuss paths, missions, and quests with your friends and avengers.
- You *must* do the actual programming yourself. Do *not* copy anyone's programs, from other students or from previous batches or using AI (without attribution).

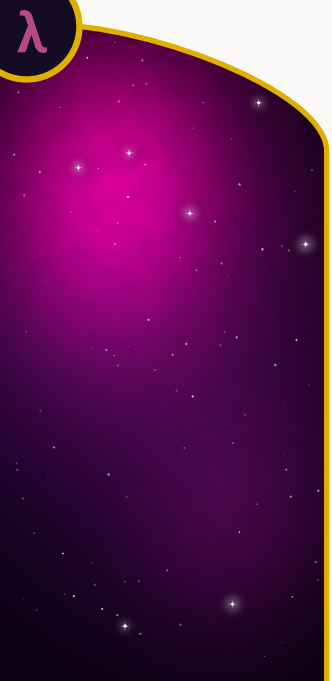
First Mission: Rune Trials

- ...will be released at 11:45 **today**.
- Missions are submitted individually and carry significant XP.
- Path P1B will be released at 11:45 today.
- You *can* discuss paths, missions, and quests with your friends and avengers.
- You *must* do the actual programming yourself. Do *not* copy anyone's programs, from other students or from previous batches or using AI (without attribution).
- Submitting other people's work as your own is a serious offence and will be severely punished by NUS.



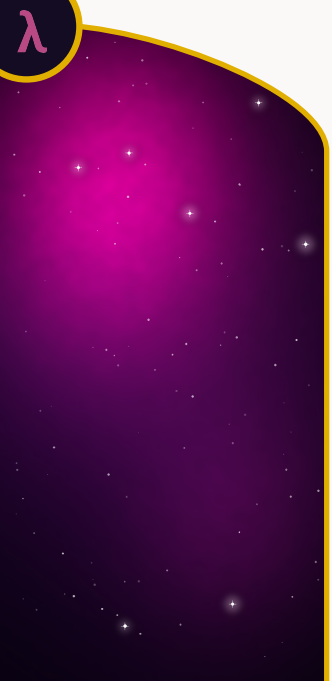


Recap: Elements of Programming (1.1)



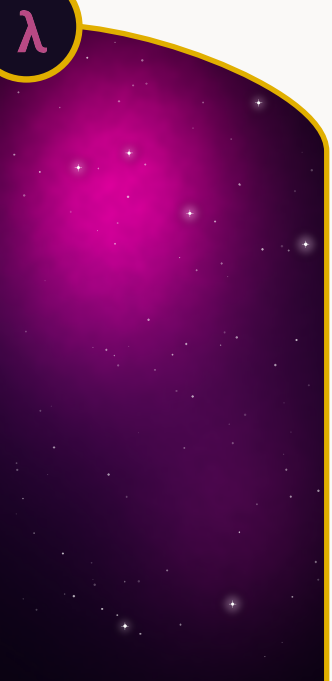
Recap: Elements of Programming (1.1)

- Primitives: things like `-42` or `False`



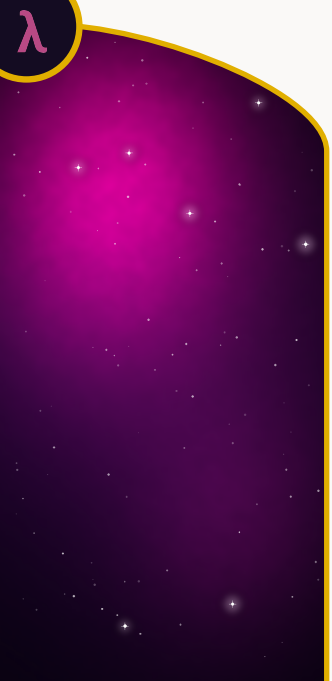
Recap: Elements of Programming (1.1)

- Primitives: things like `-42` or `False`
- Combination: things like `-42 * 7`



Recap: Elements of Programming (1.1)

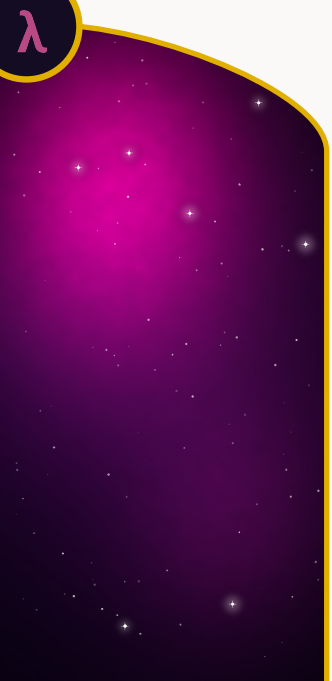
- Primitives: things like `-42` or `False`
- Combination: things like `-42 * 7`
- Name abstraction: things like `size = 2`



Recap: Elements of Programming (1.1)

- Primitives: things like `-42` or `False`
- Combination: things like `-42 * 7`
- Name abstraction: things like `size = 2`
- Conditional expressions:

```
consequent if predicate else alternative
```

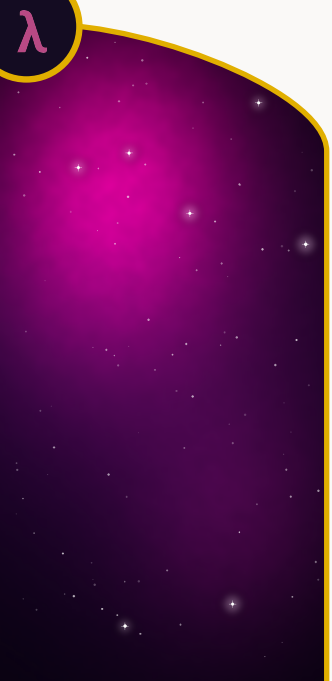


Recap: Elements of Programming (1.1)

- Primitives: things like `-42` or `False`
- Combination: things like `-42 * 7`
- Name abstraction: things like `size = 2`
- Conditional expressions:

  ```
  consequent if predicate else alternative
  ```

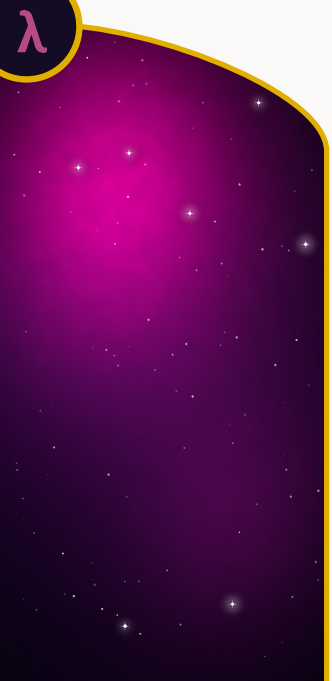
- Functional abstraction: more on this today



A function can be a “black box”

Example

`math_sqrt`



A function can be a “black box”

Example

`math_sqrt`

- Input: any number
- Output: a number whose square is the input number

A function can be a “black box”

Example

```
math_sqrt
```

- Input: any number
- Output: a number whose square is the input number

Function application

```
print(math_sqrt(15))
```

Means of Abstraction: Compound functions

Example

```
def square(x):
    return x * x
```

Means of Abstraction: Compound functions

Example

```
def square(x):
    return x * x
```

What does this statement mean?

Like declaration assignments, this *function definition* declares a name, here `square`. The value associated with the name is a function that takes an argument `x` and produces (returns) the result of multiplying `x` by itself.

Evaluation of expressions

```
1 + 2 * 3 + 4
```

Evaluation of expressions

```
1 + 2 * 3 + 4
```

Replace “eligible” subexpression with result

```
(1 + (2 * 3)) + 4
```

Evaluation of expressions

```
1 + 2 * 3 + 4
```

Replace “eligible” subexpression with result

```
(1 + (2 * 3)) + 4
```

```
(1 + 6) + 4
```

Evaluation of expressions

```
1 + 2 * 3 + 4
```

Replace “eligible” subexpression with result

```
(1 + (2 * 3)) + 4

(1 + 6) + 4

7 + 4
```

Evaluation of expressions

```
1 + 2 * 3 + 4
```

Replace “eligible” subexpression with result

```
(1 + (2 * 3)) + 4

(1 + 6) + 4

7 + 4

11
```

Evaluation of declaration assignments

```
a = 3 * 5
print(a + 5)
```

Evaluation of declaration assignments

```
a = 3 * 5
print(a + 5)
```

Evaluate what is after the $=$

```
size = 3
2 * size
```

Evaluation of declaration assignments

```
a = 3 * 5
print(a + 5)
```

Evaluate what is after the $=$

```
size = 3
2 * size
```

Then replace that name in rest of the program with that value.

```
2 * 3
```

Evaluation of declaration assignments

```
a = 3 * 5
print(a + 5)
```

Evaluate what is after the $=$

```
size = 3
2 * size
```

Then replace that name in rest of the program with that value.

```
2 * 3
```

```
6
```

Evaluation of conditional expressions

```
1 if 3 < 4 else 2
```

Evaluation of conditional expressions

```
1 if 3 < 4 else 2
```

Evaluate the predicate first.

```
1 if True else 2
```

Evaluation of conditional expressions

```
1 if 3 < 4 else 2
```

Evaluate the predicate first.

```
1 if True else 2
```

If the predicate evaluates to `True`, evaluate the consequent

```
1
```

Evaluation of conditional expressions

```
1 if 3 < 4 else 2
```

Evaluate the predicate first.

```
1 if True else 2
```

If the predicate evaluates to `True`, evaluate the consequent

```
1
```

If the predicate evaluates to `False`, evaluate the alternative

Evaluation of conditional expressions

```
1 if 3 < 4 else 2
```

Evaluate the predicate first.

```
1 if True else 2
```

If the predicate evaluates to `True`, evaluate the consequent

```
1
```

If the predicate evaluates to `False`, evaluate the alternative

```
1 if 3 > 4 else 2
```

Evaluation of conditional expressions

```
1 if 3 < 4 else 2
```

Evaluate the predicate first.

```
1 if True else 2
```

If the predicate evaluates to `True`, evaluate the consequent

```
1
```

If the predicate evaluates to `False`, evaluate the alternative

```
1 if 3 > 4 else 2
```

```
1 if False else 2
```

Evaluation of conditional expressions

```
1 if 3 < 4 else 2
```

Evaluate the predicate first.

```
1 if True else 2
```

If the predicate evaluates to `True`, evaluate the consequent

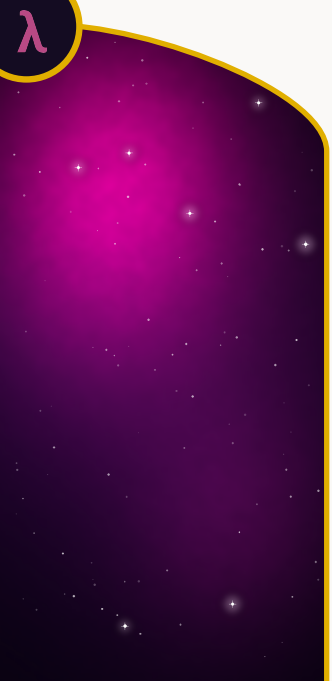
```
1
```

If the predicate evaluates to `False`, evaluate the alternative

```
1 if 3 > 4 else 2
```

```
1 if False else 2
```

```
2
```

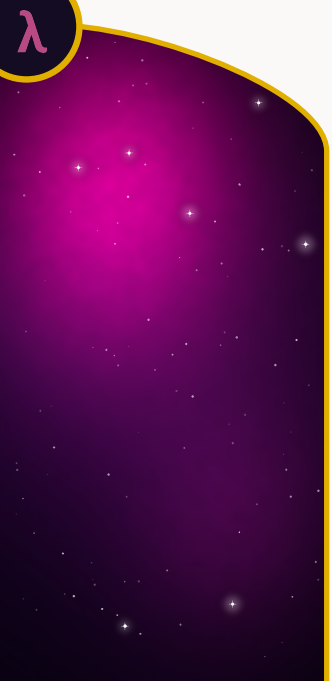


Primitives

Some simple runes such as

- `rcross`
- `sail`
- `corner`
- `nova`
- `heart`
- `show` is a primitive operation that displays a rune.

```
show(heart)
```



Where do these primitives come from?

The `rune` module

`rcross`, `sail`, `corner`, `nova`, `heart`, and `show` are not built into Python. They come from the `rune` module.

Where do these primitives come from?

The `rune` module

`rcross`, `sail`, `corner`, `nova`, `heart`, and `show` are not built into Python. They come from the `rune` module.

Import them before use

Every program that uses them needs an `import` statement first:

```
from rune import (
    show, heart, rcross, sail, nova, corner
)
```

Primitive Operations

Rune-Transformations

Functions that take a rune and produce a new rune from it

Example rune transformation

Turn a given rune a quarter turn right: `quarter_turn_right`

Example in Python

```
quarter_turn_right(quarter_turn_right(sail))
```

Abstraction: Functions and naming

```
def turn_upside_down(rune):
    return quarter_turn_right(quarter_turn_right(rune))

# example use:
show(turn_upside_down(heart))
```

Abstraction: Functions and naming (continued)

```
def quarter_turn_left(rune):
    return quarter_turn_right(turn_upside_down(rune))

# example use:
show(quarter_turn_left(heart))
```

Combination operation: stacking

```
stack(rcross, sail)
```

Your turn: Combination abstraction

```
def beside(rune1, rune2):
    ...

# example use:
show(beside(rcross, sail))
# should show rcross on the left
#           and sail on the right
```

Solution: Combination abstraction

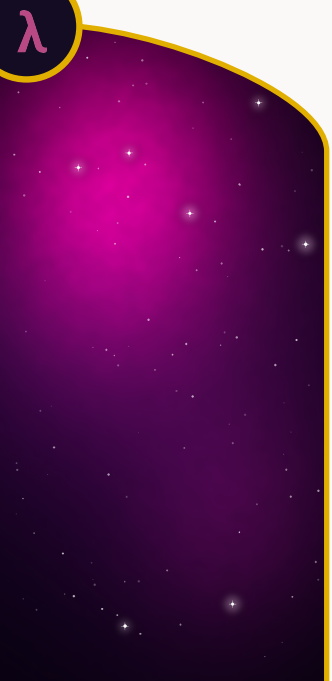
```
def beside(rune1, rune2):
    return quarter_turn_left(
        stack(
            quarter_turn_right(rune1),
            quarter_turn_right(rune2)
        )
    )
```

Solution: Combination abstraction

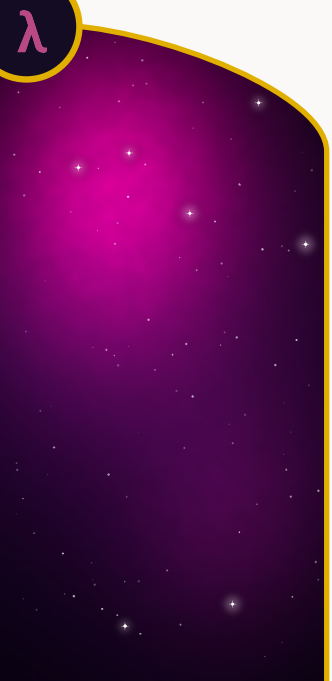
```
def beside(rune1, rune2):
    return quarter_turn_left(
        stack(
            quarter_turn_right(rune1),
            quarter_turn_right(rune2)
        )
    )
```

Good news

`beside` is important and it's already pre-defined for you in the `rune` module!

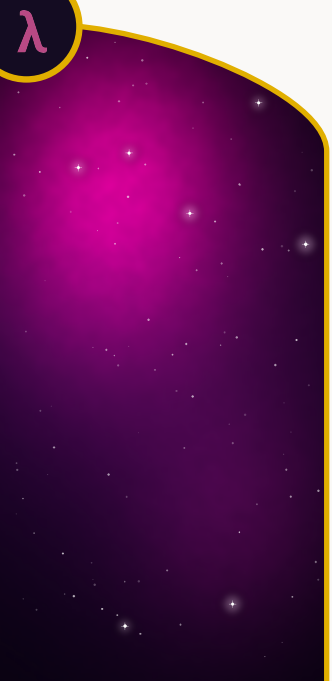


Time for the Check-In!



Time for the Check-In!

Let's DDOS the Source Academy!



Time for the Check-In!

Let's DDOS the Source Academy!

Look for Check-In I1B in the Source Academy!

Time for the Check-In!

Let's DDOS the Source Academy!

Look for Check-In I1B in the Source Academy!

(Only your best 18 of 20 Check-Ins count, so you may miss up to two without penalty.)

Combination: stacking something n times

```
stackn(5, heart)
```

Lecture L2A

We will define `stackn` in Python, using rune primitives.

Naming and more combination: rectangular quilting

```
my_quilt = stackn(
    5,
    quarter_turn_right(stackn(7,
        quarter_turn_left(heart)))
)
```

Abstraction: rectangular quilting

```
def quilt(n, m, rune):
    return stackn(
        n,
        quarter_turn_right(stackn(m,
            quarter_turn_left(rune)))
    )
```

Combination: a more complex rune

```
stack(
    beside(quarter_turn_right(rcross),
        turn_upside_down(rcross)),
    beside(rcross,
        quarter_turn_left(rcross))
)
```

Naming interesting things

```
my_cross = stack(
    beside(quarter_turn_right(rcross),
        turn_upside_down(rcross)),
    beside(rcross,
        quarter_turn_left(rcross))
)
```

Abstraction: making a cross from any rune

```
def make_cross(rune):
    return stack(
        beside(quarter_turn_right(rune),
            turn_upside_down(rune)),
        beside(rune,
            quarter_turn_left(rune))
    )
```

Abstraction: making a cross from any rune

```
def make_cross(rune):
    return stack(
        beside(quarter_turn_right(rune),
            turn_upside_down(rune)),
        beside(rune,
            quarter_turn_left(rune))
    )
```

`make_cross` is a transformation.

Combination: repeating a transformation

```
show(make_cross(make_cross(nova)))
```

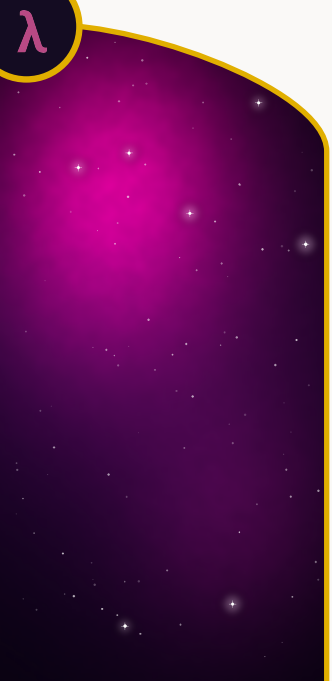
Abstraction: repeating a transformation n times

Pre-defined in module `repeat`:

```
show(repeat_apply(make_cross, 5, rcross))
```

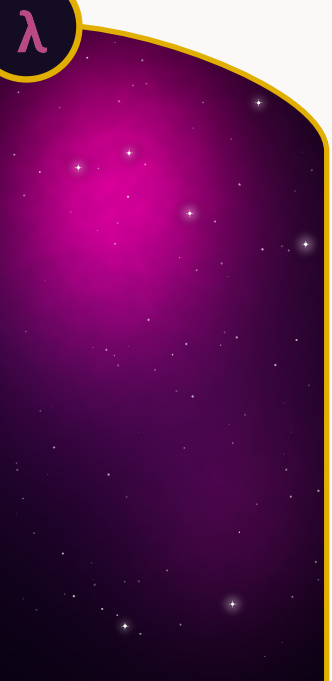
Lecture L2A

Like `stackn`, we will define this abstraction in Python next Wednesday!



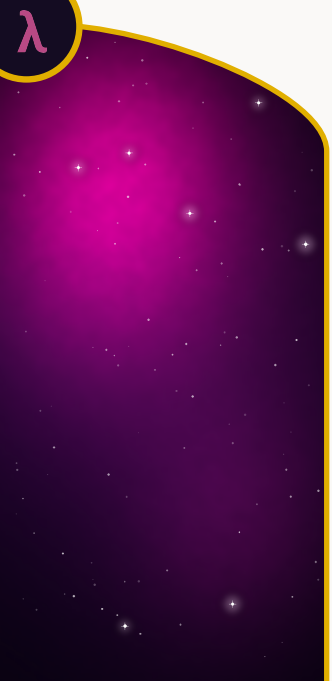
Recap

- We started with primitive constants.



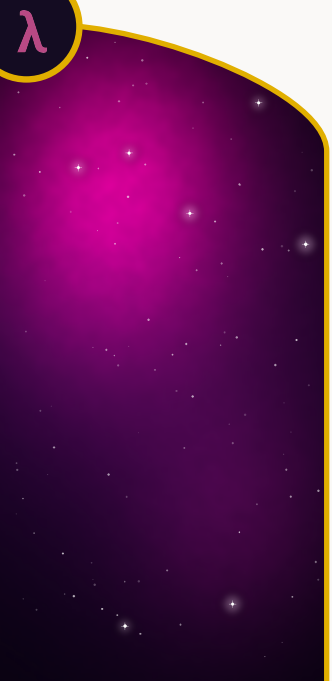
Recap

- We started with primitive constants.
- No idea how they are rendered!



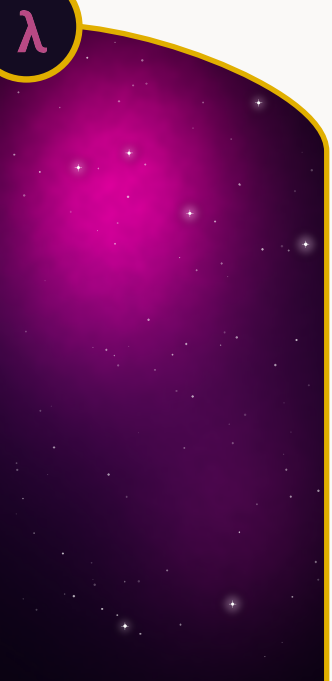
Recap

- We started with primitive constants.
- No idea how they are rendered!
  Example: `heart`



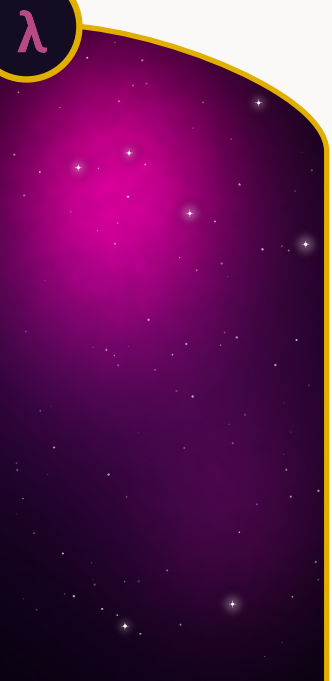
Recap

- We started with primitive constants.
- No idea how they are rendered!
  Example: `heart`
- We introduced primitive combinations.



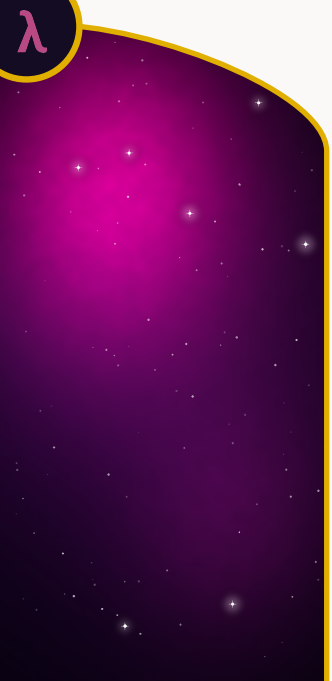
Recap

- We started with primitive constants.
- No idea how they are rendered!
  Example: `heart`
- We introduced primitive combinations.
- No idea how the primitive combinations work!



Recap

- We started with primitive constants.
- No idea how they are rendered!
  Example: `heart`
- We introduced primitive combinations.
- No idea how the primitive combinations work!
  Example: `quarter_turn_right`

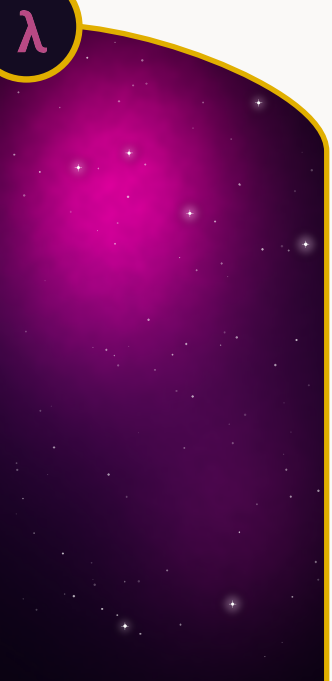


Recap

- We started with primitive constants.
- No idea how they are rendered!
  Example: `heart`
- We introduced primitive combinations.
- No idea how the primitive combinations work!
  Example: `quarter_turn_right`
- Yet we can use primitives and combinations to generate complex runes.

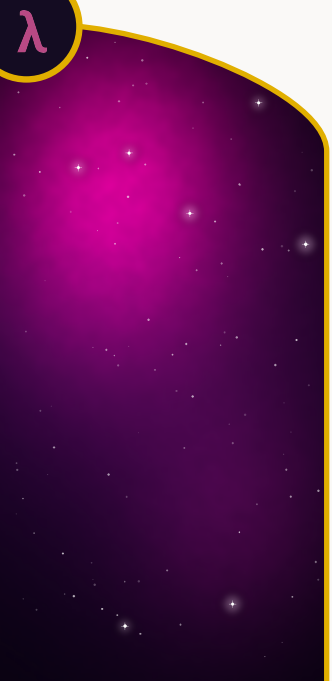
Recap

- We started with primitive constants.
- No idea how they are rendered!
  Example: `heart`
- We introduced primitive combinations.
- No idea how the primitive combinations work!
  Example: `quarter_turn_right`
- Yet we can use primitives and combinations to generate complex runes.
  - **Abstractions** allow us to master complexity:



Recap

- We started with primitive constants.
- No idea how they are rendered!
  Example: `heart`
- We introduced primitive combinations.
- No idea how the primitive combinations work!
  Example: `quarter_turn_right`
- Yet we can use primitives and combinations to generate complex runes.
  - **Abstractions** allow us to master complexity:
    Naming



Recap

- We started with primitive constants.
- No idea how they are rendered!
  Example: `heart`
- We introduced primitive combinations.
- No idea how the primitive combinations work!
  Example: `quarter_turn_right`
- Yet we can use primitives and combinations to generate complex runes.
  - **Abstractions** allow us to master complexity:
    Naming and **functional abstraction**

Happy Half Valentine's Day!

```
from rune import show, heart,
    translate, rotate, red,
    pink, overlay, scale

a = translate(-0.18, 0.02,
    rotate(0.3,
    red(scale(0.42, heart))))
b = translate(0.18, -0.02,
    rotate(0.3,
    pink(scale(0.42, heart))))

show(overlay(a, b))
```